

TK

TheKnife

Manuale Tecnico

Documentazione architettuale e implementativa

Laboratorio Interdisciplinare B — a.a. 2025/2026
Università degli Studi dell'Insubria

Matteo Vigano — matricola 760537 — sede CO
Fabio Vecaj — matricola 761232 — sede CO
De Zuane Samuele — matricola 763267 — sede CO

Versione 2.0 — Maggio 2025

Indice

1. Architettura del sistema

- 1.1 Architettura client-server
- 1.2 Struttura Maven multi-modulo
- 1.3 Diagramma dei moduli

2. Modulo theknife-common

- 2.1 Classi di modello
- 2.2 Protocollo di comunicazione
- 2.3 CommandType

3. Modulo theknife-server

- 3.1 ServerTK – punto di ingresso
- 3.2 ClientHandler – gestione sessione
- 3.3 DatabaseManager – JDBC
- 3.4 Gestione concorrenza

4. Modulo theknife-client

- 4.1 ClientTK – main e sessione
- 4.2 ServerConnection – comunicazione
- 4.3 Sistema di design UITheme
- 4.4 FancyFrame – finestra principale
- 4.5 Pannelli funzionali

5. Database PostgreSQL

- 5.1 Schema ER concettuale
- 5.2 Schema ristrutturato
- 5.3 Tabelle e vincoli
- 5.4 Script di inizializzazione

6. Sicurezza

- 6.1 Cifratura password SHA-256
- 6.2 Autenticazione lato server

7. Compilazione e avvio

- 7.1 Prerequisiti
- 7.2 Comandi Maven
- 7.3 Avvio server e client

8. Scelte tecnologiche

9. Diagramma di sequenza

10. Struttura directory progetto

1 Architettura del sistema

1.1 Architettura client-server

TheKnife adotta un'architettura client-server distribuita a due livelli. Il client (**clientTK.jar**) implementa la GUI Swing e comunica con il server tramite socket TCP. Il server (**serverTK.jar**) gestisce la logica di business e l'accesso al database PostgreSQL tramite JDBC.

La comunicazione avviene tramite oggetti Java serializzati (*Serializable*) trasmessi su `ObjectOutputStream/ObjectInputStream`. Il server mantiene una connessione persistente per client, conservando lo stato di autenticazione per tutta la sessione.

Componente	Ruolo	Tecnologia
clientTK.jar	Interfaccia grafica, invio richieste	Java 17, Swing, Socket TCP
serverTK.jar	Logica business, accesso DB	Java 17, JDBC, Thread pool
PostgreSQL	Persistenza dati	PostgreSQL 14+, driver 42.7.3

1.2 Struttura Maven multi-modulo

Il progetto è strutturato come Maven multi-modulo con un POM padre (**theknife-parent**) che centralizza le versioni delle dipendenze. I tre sotto-moduli sono: **theknife-common**, **theknife-server**, **theknife-client**.

```
VIGANO_760537/
■■■ pom.xml                ← POM padre
■■■ theknife-common/       ← modello + protocollo condivisi
■   ■■■ src/main/java/.../common/
■       ■■■ CommandType.java
■       ■■■ Request.java
■       ■■■ Response.java
■       ■■■ model/ (Ristorante, Utente, Recensione)
■■■ theknife-server/       ← backend JDBC
■   ■■■ src/main/java/.../server/
■       ■■■ ServerTK.java
■       ■■■ ClientHandler.java
■       ■■■ DatabaseManager.java
■■■ theknife-client/       ← GUI Swing
■   ■■■ src/main/java/.../client/
■       ■■■ ClientTK.java
■       ■■■ ServerConnection.java
■       ■■■ gui/ (UITheme, FancyFrame, panels/...)
■■■ bin/
■   ■■■ serverTK.jar
■   ■■■ clientTK.jar
■■■ src/db/
    ■■■ init.sql
```

```
data.sql
```

1.3 Diagramma dei moduli

Il modulo **theknife-common** è una dipendenza sia del server che del client: contiene le classi condivise (modello di dominio e protocollo) che devono essere identiche su entrambi i lati per garantire la serializzazione.

Modulo	Dipende da	Produce
theknife-common	— (nessuna dipendenza interna)	theknife-common-1.0.0.jar
theknife-server	theknife-common	serverTK.jar (fat-JAR)
theknife-client	theknife-common	clientTK.jar (fat-JAR)

2 Modulo theknife-common

2.1 Classi di modello

Il package `it.uninsubria.theknife.common.model` contiene le tre classi che rappresentano il dominio dell'applicazione. Tutte implementano `java.io.Serializable` per la trasmissione via socket.

Classe	Campi principali	Note
Ristorante	nome, nazione, citta, indirizzo, latitudine, longitudine, fasciaPrezzo, delivery, prenotazione, tipoCucina	mediaStelle e numeroRecensioni calcolati dal server
Utente	nome, cognome, username, passwordHash, dataNascita, domicilio, ruolo	passwordHash = SHA-256 della password
Recensione	nomeRistorante, usernameCliente, stelle (1-5), testo, risposta	risposta è null se il ristorante non ha ancora risposto

2.2 Protocollo di comunicazione

La comunicazione tra client e server si basa su due classi serializzabili: **Request** (client → server) e **Response** (server → client). Entrambe vengono trasmesse tramite `ObjectOutputStream` su socket TCP.

Classe Request

```
public class Request implements Serializable {
    private CommandType comando;
    private String usernameLoggato; // null se guest
    private Map<String, Object> parametri;

    public Request aggiungiParametro(String chiave, Object valore) {
        parametri.put(chiave, valore);
        return this; // fluent API
    }
}
```

Classe Response

```
public class Response implements Serializable {
    private boolean successo;
    private String messaggio;
    private Object dato; // cast con getDatoTipizzato()

    public static Response ok(Object dato) { ... }
    public static Response errore(String msg) { ... }

    @SuppressWarnings("unchecked")
    public <T> T getDatoTipizzato() { return (T) dato; }
}
```

2.3 CommandType

L'enumerazione **CommandType** centralizza tutti i comandi del protocollo. Il server effettua un pattern matching (*switch expression* Java 17) sul tipo di comando e instrada la logica al metodo corretto del `DatabaseManager`.

Categoria	Comandi
Guest (no login)	CERCA_RISTORANTI, VISUALIZZA_RECENSIONI
Autenticazione	LOGIN, REGISTRAZIONE
Cliente	CLIENTE_VISUALIZZA_PREFERITI, CLIENTE_AGGIUNGI_PREFERITO, CLIENTE_RIMUOVI_PREFERITO, CLIENTE_AGGIUNGI_RECENSIONE, CLIENTE_MODIFICA_RECENSIONE, CLIENTE_ELIMINA_RECENSIONE
Ristoratore	RISTORATORE_AGGIUNGI_RISTORANTE, RISTORATORE_VISUALIZZA_RIEPILOGO, RISTORATORE_RISPONDI_RECENSIONE

3 Modulo theknife-server

3.1 ServerTK – punto di ingresso

La classe **ServerTK** è il punto di ingresso del server. All'avvio chiede interattivamente host PostgreSQL, porta DB, password e porta di ascolto TCP. Crea un **ExecutorService** a pool fisso (50 thread) e per ogni client accettato crea un **ClientHandler** che viene sottomesso al pool.

```
// ServerTK.java - ciclo principale
ExecutorService pool = Executors.newFixedThreadPool(50);
ServerSocket server = new ServerSocket(portaServer);
while (true) {
    Socket client = server.accept();
    pool.submit(new ClientHandler(client, dbConfig));
}
```

3.2 ClientHandler – gestione sessione

ClientHandler implementa **Runnable**. Ogni istanza gestisce la comunicazione con un singolo client per tutta la durata della connessione. Il campo **utenteLoggato** mantiene la sessione: viene impostato al login e azzerato al logout o alla chiusura della connessione.

```
public class ClientHandler implements Runnable {
    private final Socket socket;
    private Utente utenteLoggato = null; // sessione per-thread
    @Override public void run() {
        try (DatabaseManager db = new DatabaseManager(dbConfig)) {
            ObjectInputStream in = new ObjectInputStream(socket.getInputStream());
            ObjectOutputStream out = new ObjectOutputStream(socket.getOutputStream());
            while (true) {
                Request req = (Request) in.readObject();
                Response res = elabora(req, db);
                out.writeObject(res);
                out.reset(); // evita caching oggetti
            }
        }
    }
    private Response elabora(Request req, DatabaseManager db) {
        return switch (req.getComando()) {
            case LOGIN -> { utenteLoggato = db.login(...); yield Response.ok(utenteLoggato);
        }

            case CERCA_RISTORANTI -> Response.ok(db.cercaRistoranti(...));
            // ... tutti gli altri comandi
        };
    }
}
```

3.3 DatabaseManager – JDBC

DatabaseManager implementa **AutoCloseable**. Ogni istanza ha la propria connessione JDBC (pattern *una connessione per thread*). Usa **PreparedStatement** per tutte le query, prevenendo SQL injection.

Metodo	Query SQL	Ritorna
login(user, hash)	SELECT * FROM utenti WHERE username=? AND password_hash=?	Utente o null
registrazione(...)	INSERT INTO utenti VALUES (?, ?, ?, ?, ?, ?)	Boolean
cercaRistoranti(filtri)	SELECT r.*, AVG(rec.stelle), COUNT(rec.*) FROM ristoranti r LEFT JOIN recensioni	List
aggiungiPreferito(u,r)	INSERT INTO preferiti VALUES (?, ?)	Boolean
aggiungiRecensione(...)	INSERT INTO recensioni VALUES (?, ?, ?, ?)	Boolean
modificaRecensione(...)	UPDATE recensioni SET stelle=?, testo=? WHERE ...	Boolean
eliminaRecensione(...)	DELETE FROM recensioni WHERE ...	Boolean
rispondiRecensione(...)	UPDATE recensioni SET risposta=? WHERE ...	Boolean
riepilogoRistorante(u)	SELECT r.*, AVG, COUNT FROM ristoranti r WHERE proprietario=?	List

3.4 Gestione concorrenza

La concorrenza è gestita con il pattern *una connessione JDBC per thread*: ogni ClientHandler apre la propria istanza di DatabaseManager. Questo elimina le race condition sulle risorse condivise senza necessità di sincronizzazione esplicita.

- Nessun shared state tra thread diversi (ogni thread ha il proprio DatabaseManager)
- ExecutorService gestisce il ciclo di vita del thread pool (shutdown graceful)
- DatabaseManager implementa AutoCloseable → chiusura automatica con try-with-resources
- out.reset() dopo ogni risposta evita il caching degli oggetti nell'ObjectOutputStream

4 Modulo theknife-client

4.1 ClientTK – main e sessione

ClientTK è la classe main del client. Gestisce la sessione utente tramite campo statico (**utenteLoggato**) accessibile da tutti i pannelli. Crea la **ServerConnection** e avvia la **FancyFrame** sull'EDT di Swing.

```
public class ClientTK {
    private static Utente utenteLoggato;
    private static ServerConnection connessione;

    public static void main(String[] args) {
        // Dialog connessione: host e porta
        connessione = new ServerConnection(host, porta);
        SwingUtilities.invokeLater(() -> new FancyFrame().setVisible(true));
    }

    public static boolean isLoggato() { return utenteLoggato != null; }
    public static Utente getUtenteLoggato() { return utenteLoggato; }
    public static void setUtenteLoggato(Utente u) { utenteLoggato = u; }
    public static void logout() { utenteLoggato = null; }
    public static ServerConnection getConnessione() { return connessione; }
}
```

4.2 ServerConnection – comunicazione

ServerConnection gestisce il socket TCP verso il server. Il metodo **invia()** invia una Request e attende la Response sincrona. La password viene hashata SHA-256 prima di qualsiasi trasmissione.

```
public Response invia(Request req) throws IOException, ClassNotFoundException {
    out.writeObject(req);
    out.flush();
    out.reset();
    return (Response) in.readObject();
}

public static String sha256(String password) {
    MessageDigest md = MessageDigest.getInstance("SHA-256");
    byte[] hash = md.digest(password.getBytes(StandardCharsets.UTF_8));
    StringBuilder sb = new StringBuilder();
    for (byte b : hash) sb.append(String.format("%02x", b));
    return sb.toString(); // 64 caratteri esadecimali
}
```

4.3 Sistema di design UITheme

UITheme è la classe centrale del sistema di design. Centralizza tutti i colori, font, spaziature e factory di componenti. Tutti i componenti custom usano Java2D puro (nessuna dipendenza da font Unicode).

Componente	Classe	Funzionalità
TKButton	UITheme.TKButton extends JButton	Hover animato via Timer, sfondo pieno o ghost
CardPanel	UITheme.CardPanel extends JPanel	Ombra a 2 strati, bordo arrotondato, hover
StarChip	UITheme.StarChip extends JButton	Stelle disegnate Java2D, toggle attivo/inattivo
SideBarItem	UITheme.SideBarItem extends JButton	Indicatore oro animato, hover fluido
dialogHeader	Metodo factory	Header navy affidabile (setBackground+setOpaque)
starLabel	Metodo factory	Stelle Java2D + testo, no Unicode
rh(Graphics g)	Metodo statico	Crea Graphics2D con antialiasing attivato
drawStar(...)	Metodo statico pubblico	Stella a 5 punte, parametri configurabili

4.4 FancyFrame – finestra principale

FancyFrame è il JFrame principale. Contiene la top bar, la sidebar e un'area centrale con **CardLayout** che gestisce la navigazione tra pannelli.

```
// Costanti card di navigazione
public static final String CARD_HOME      = "Home";
public static final String CARD_SEARCH    = "Search";
public static final String CARD_RISTORANTI = "Ristoranti";
public static final String CARD_RECENSIONI = "Recensioni";
public static final String CARD_PREFERITI = "Preferiti";
public static final String CARD_DETAIL    = "Detail";

// Navigazione
public void showCard(String name) { cardLayout.show(centerPanel, name); }
```

4.5 Pannelli funzionali

Ogni pannello estende **GradientPanel** e gestisce la propria logica di presentazione. Le chiamate al server vengono eseguite in **SwingWorker** per non bloccare l'EDT.

Pannello	Classe	Funzionalità principali
Home	HomePanel	Hero section, ricerca per città/domicilio, feature card, CTA
Esplora	SearchPanel	Ricerca avanzata con filtri, chip attivi rimovibili, griglia card
Dettaglio	RestaurantDetailPanel	Info ristorante, stelle, badge servizi, lista recensioni
Preferiti	PreferitiPanel	Griglia preferiti, rimozione rapida ×, ordinamento toggle
Recensioni	RecensioniPanel	Filtro stelle Java2D, CRUD recensioni, dialog risposta
Miei locali	RistorantiPanel	Griglia locali, statistiche, dialog aggiunta ristorante

NOTA

Tutti i pannelli usano SwingWorker per le chiamate di rete, garantendo la reattività dell'interfaccia.

5 Database PostgreSQL

5.1 Schema ER concettuale

Lo schema concettuale identifica quattro entità principali: **UTENTE**, **RISTORANTE**, **RECENSIONE** e **PREFERITO**. L'entità **UTENTE** ha un attributo discriminante 'ruolo' (cliente/ristoratore). **RECENSIONE** è un'entità debole identificata dalla coppia (ristorante, cliente). **PREFERITO** è un'associazione binaria tra cliente e ristorante.

5.2 Schema ristrutturato

In fase di ristrutturazione sono state effettuate le seguenti scelte:

- L'entità debole **PREFERITO** è tradotta in una tabella di associazione con PK composta (username, nome_ristorante)
- La risposta del ristorante è mantenuta come attributo nullable nella tabella **RECENSIONI**, eliminando la necessità di una tabella separata
- Il ruolo dell'utente (cliente/ristoratore) è implementato come attributo stringa con CHECK constraint invece della generalizzazione
- Le chiavi esterne hanno ON DELETE CASCADE per garantire l'integrità referenziale

5.3 Tabelle e vincoli

```
-- Tabella utenti
CREATE TABLE utenti (
    username      VARCHAR(50) PRIMARY KEY,
    nome          VARCHAR(50) NOT NULL,
    cognome       VARCHAR(50) NOT NULL,
    password_hash CHAR(64)    NOT NULL, -- SHA-256
    data_nascita  DATE,
    domicilio     VARCHAR(100),
    ruolo         VARCHAR(20) NOT NULL
                CHECK (ruolo IN ('cliente', 'ristoratore'))
);

-- Tabella ristoranti
CREATE TABLE ristoranti (
    nome          VARCHAR(100) PRIMARY KEY,
    nazione       VARCHAR(50) NOT NULL,
    citta         VARCHAR(100) NOT NULL,
    indirizzo     VARCHAR(200),
    latitudine    FLOAT CHECK (latitudine BETWEEN -90 AND 90),
    longitudine   FLOAT CHECK (longitudine BETWEEN -180 AND 180),
    fascia_prezzo NUMERIC(8,2) NOT NULL CHECK (fascia_prezzo > 0),
    delivery      BOOLEAN DEFAULT FALSE,
    prenotazione  BOOLEAN DEFAULT FALSE,
    tipo_cucina   VARCHAR(100) NOT NULL,
```

```
    proprietario VARCHAR(50) REFERENCES utenti(username) ON DELETE CASCADE
);
-- Tabella recensioni
CREATE TABLE recensioni (
    nome_ristorante VARCHAR(100) REFERENCES ristoranti(nome) ON DELETE CASCADE,
    username_cliente VARCHAR(50) REFERENCES utenti(username) ON DELETE CASCADE,
    stelle          SMALLINT      NOT NULL CHECK (stelle BETWEEN 1 AND 5),
    testo           TEXT          NOT NULL,
    risposta        TEXT, -- NULL se non risposto
    PRIMARY KEY (nome_ristorante, username_cliente)
);
-- Tabella preferiti
CREATE TABLE preferiti (
    username          VARCHAR(50) REFERENCES utenti(username) ON DELETE CASCADE,
    nome_ristorante   VARCHAR(100) REFERENCES ristoranti(nome) ON DELETE CASCADE,
    PRIMARY KEY (username, nome_ristorante)
);
```

5.4 Script di inizializzazione

Il database viene inizializzato con due script SQL:

- **init.sql**: crea tutte le tabelle con vincoli CHECK, indici e chiavi esterne. Può essere eseguito tramite psql o pgAdmin.
- **data.sql**: popola il database con dati di test (6 utenti, 9 ristoranti, 9 recensioni, 9 preferiti). Tutti gli utenti test hanno password *password123*.

```
-- Esecuzione da terminale:
psql -U postgres -d theknife -f src/db/init.sql
psql -U postgres -d theknife -f src/db/data.sql

-- Oppure tramite Maven:
mvn -pl theknife-server exec:exec@init-db
```

6 Sicurezza

6.1 Cifratura password SHA-256

La password non viene mai trasmessa in chiaro sulla rete. Il client calcola l'hash SHA-256 della password prima di inviarla al server.

```
// Lato client - ServerConnection.java
public static String sha256(String password) {
    MessageDigest md = MessageDigest.getInstance("SHA-256");
    byte[] hashBytes = md.digest(password.getBytes(StandardCharsets.UTF_8));
    StringBuilder hexStr = new StringBuilder();
    for (byte b : hashBytes)
        hexStr.append(String.format("%02x", b));
    return hexStr.toString(); // es: "ef92b778bafe771e..."
}
// Il server confronta direttamente gli hash:
// SELECT * FROM utenti WHERE username=? AND password_hash=?
```

L'hash SHA-256 produce sempre una stringa di 64 caratteri esadecimali. Il campo **password_hash** nella tabella utenti è definito CHAR(64). La password in chiaro non transita mai sulla rete e non viene mai memorizzata.

6.2 Autenticazione lato server

L'autenticazione viene verificata sul server ad ogni operazione riservata. **ClientHandler** mantiene il campo **utenteLoggato** per thread: se un comando richiede autenticazione e utenteLoggato è null, il server risponde con Response.errore(). Il ruolo viene verificato prima di eseguire operazioni riservate.

Verifica	Quando	Come
Utente loggato	Ogni comando non-guest	utenteLoggato != null
Ruolo cliente	Preferiti, recensioni CRUD	utenteLoggato.isCliente()
Ruolo ristoratore	Gestione locali, risposte	utenteLoggato.isRistoratore()
Proprietario ristorante	Risposta recensione	ristorante.getProprietario().equals(username)

7 Compilazione e avvio

7.1 Prerequisiti

Componente	Versione	Note
Java JDK	17 LTS (minimo)	Versioni superiori compatibili
Apache Maven	3.8+	Gestione dipendenze e build
PostgreSQL	14+	Database del server
JDBC Driver PostgreSQL	42.7.3	Incluso nel fat-JAR del server

7.2 Comandi Maven

```
# Compilare tutti i moduli e generare i JAR
mvn clean package

# Generare la documentazione JavaDoc
mvn javadoc:aggregate

# Output: target/site/apidocs/index.html

# Inizializzare il database (richiede PostgreSQL in esecuzione)
mvn -pl theknife-server exec:exec@init-db

# Compilare solo il client (dopo aver compilato common)
mvn -pl theknife-client package

# Eseguire i test (se presenti)
mvn test
```

7.3 Avvio server e client

```
# 1. Avviare il server (richiede PostgreSQL in esecuzione)
java -jar bin\serverTK.jar

# Il server chiede: host DB, porta DB, password DB, porta di ascolto
# Valori tipici: localhost, 5432, [password], 9090

# 2. Avviare il client (in un altro terminale)
java -jar bin\clientTK.jar

# Il client chiede: host server, porta server
# Valori tipici: localhost, 9090

# Nota su Windows: usare backslash (\) o percorsi tra virgolette
```

NOTA

Il server deve essere avviato PRIMA del client. La connessione è persistente per tutta la sessione.

8 Scelte tecnologiche

Tecnologia	Versione	Motivazione della scelta
Java	17 LTS	Multiplatforma, switch expressions, record, text blocks
PostgreSQL	14+	Richiesto dalle specifiche, ACID, performance
JDBC Driver	42.7.3	Accesso nativo PostgreSQL, PreparedStatement per SQL injection prevention
Maven	3.8+	Build reproducibile, gestione dipendenze, multi-modulo, JavaDoc
Java Swing	JDK built-in	GUI richiesta dalle specifiche, nessuna dipendenza esterna
SHA-256	JDK built-in	Cifratura password senza librerie esterne
Java Serialization	JDK built-in	Trasmissione oggetti su socket, compatibilità automatica
ObjectOutputStream	JDK built-in	Stream bidirezionale su socket TCP, semplicità
ExecutorService	JDK built-in	Thread pool gestito, shutdown graceful
SwingWorker	JDK built-in	Operazioni asincrone senza bloccare l'EDT di Swing
Java2D (Graphics2D)	JDK built-in	Icone e componenti custom senza dipendenza da font Unicode
GridBagLayout	JDK built-in	Layout flessibile per form e pannelli complessi

Scelte architetturali notevoli

- **Una connessione JDBC per thread:** eliminazione race condition senza synchronized espliciti
- **Sessione server-side:** lo stato di login è mantenuto in ClientHandler (non inviato ad ogni richiesta)
- **Hash lato client:** la password in chiaro non transita mai sulla rete, neanche durante il login
- **API fluente per Request:** aggiungiParametro() ritorna this, permettendo chain calls
- **Fat-JAR con Maven Assembly:** i JAR finali includono tutte le dipendenze, facilitando la distribuzione
- **Grafica con zero font Unicode:** tutte le icone e decorazioni grafiche usano Java2D puro

9 Diagramma di sequenza

Scenario 1: Login utente

#	Attore	Messaggio / Azione	Direzione
1	Utente	Inserisce username e password nel LoginDialog	→ GUI
2	GUI	Calcola SHA-256(password)	→ ServerConn.
3	ServerConnection	new Request(LOGIN).aggiungiParametro("username",u) .aggiungiParametro("hash",h)	→ Server TCP
4	ClientHandler	Riceve Request, chiama db.login(username, hash)	→ DB Mgr
5	DatabaseManager	SELECT * FROM utenti WHERE username=? AND password_hash=?	→ PostgreSQL
6	PostgreSQL	ResultSet con dati utente	→ DB Mgr
7	DatabaseManager	Costruisce oggetto Utente, setta utenteLoggato	→ ClientHandler
8	ClientHandler	Response.ok(utente)	→ Client TCP
9	ServerConnection	Ritorna Response al chiamante	→ GUI
10	GUI	ClientTK.setUtenteLoggato(u), aggiorna top bar	→ FancyFrame

Scenario 2: Cerca ristoranti con filtri

#	Attore	Messaggio / Azione	Direzione
1	Utente	Inserisce città, seleziona filtri, preme Cerca	→ SearchPanel
2	SearchPanel	Crea Request(CERCA_RISTORANTI) con parametri filtro	→ SwingWorker
3	SwingWorker	Esegue in background: connessione.invia(req)	→ ServerConn.

#	Attore	Messaggio / Azione	Direzione
4	ServerConnection	Serializza Request su ObjectOutputStream	→ Server TCP
5	ClientHandler	Riceve Request, chiama db.cercaRistoranti(filtri)	→ DB Mgr
6	DatabaseManager	SELECT dinamico con JOIN e WHERE clause	→ PostgreSQL
7	PostgreSQL	ResultSet con ristoranti filtrati	→ DB Mgr
8	DatabaseManager	List con mediaStelle e numRecensioni	→ ClientHandler
9	ClientHandler	Response.ok(lista)	→ Socket TCP
10	SwingWorker	done(): aggiorna griglia card sul EDT	→ SearchPanel

Scenario 3: Inserimento recensione

#	Attore	Messaggio / Azione	Direzione
1	Cliente	Apre RecensioneDialog, imposta stelle e testo	→ GUI
2	GUI	Request(CLIENTE_AGGIUNGI_RECENSIONE, username)	→ ServerConn.
3	ServerConnection	Serializza e trasmette	→ Server TCP
4	ClientHandler	Verifica utenteLoggato.isCliente()	→ DB Mgr
5	DatabaseManager	INSERT INTO recensioni VALUES (?, ?, ?, ?) stelle CHECK 1-5	→ PostgreSQL
6	PostgreSQL	Conferma inserimento	→ DB Mgr
7	ClientHandler	Response.ok("Recensione inserita con successo")	→ Client
8	GUI	Mostra messaggio OK, ricarica lista recensioni	→ DetailPanel

10 Struttura directory progetto

```

VIGANO_760537/
├── pom.xml                ← POM padre (versioni centralizzate)
├── autori.txt             ← Autori e matricole
├── README.md             ← Istruzioni build e avvio
├── run-TheKnife.bat      ← Script avvio Windows
├──
├── bin/
│   ├── serverTK.jar      ← Fat-JAR server
│   └── clientTK.jar      ← Fat-JAR client
│   └──
├── theknife-common/
│   ├── pom.xml
│   └── src/main/java/it/uninsubria/theknife/common/
│       ├── CommandType.java
│       ├── Request.java
│       ├── Response.java
│       └── model/
│           ├── Ristorante.java
│           ├── Utente.java
│           └── Recensione.java
│       └──
├── theknife-server/
│   ├── pom.xml
│   └── src/main/java/it/uninsubria/theknife/server/
│       ├── ServerTK.java
│       ├── ClientHandler.java
│       └── DatabaseManager.java
│       └──
├── theknife-client/
│   ├── pom.xml
│   └── src/main/java/it/uninsubria/theknife/client/
│       ├── ClientTK.java
│       ├── ServerConnection.java
│       └── gui/
│           ├── UITheme.java
│           ├── FancyFrame.java
│           ├── GradientPanel.java
│           ├── LoginDialog.java
│           ├── RegisterDialog.java
│           └── panels/
│               ├── HomePanel.java
│               ├── SearchPanel.java
│               ├── RestaurantDetailPanel.java
│               ├── RecensioniPanel.java
│               ├── PreferitiPanel.java
│               └── RistorantiPanel.java
│           └──
├── src/db/

```

```
■ ■■■■ init.sql           ← Schema database
■ ■■■■ data.sql          ← Dati di test
■
■■■■ doc/
  ■■■■ manuale-utente.pdf
  ■■■■ manuale-tecnico.pdf
  ■■■■ javadoc/           ← mvn javadoc:aggregate
```